Rockchip RV1126B RT-Thread MCU 快速开发指南

文档标识：RK-JC-YF-A49

发布版本：V1.0.0

日期：2025-06-05

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 **© 2025** 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文主要描述了RV1126B RT-Thread MCU的基本使用方法，旨在帮助开发者快速了解并使用RT-Thread开发包。

RV1126B内置了一个High Performance MCU（HPMCU），RISCV架构，具有16K I/D Cache以及独立的中断控制单元（INTMUX）。它能够访问RV1126B的大部分低速外设（如I2C、SARADC、PWM、ADC、SPI等）、GPIO，与A53核心可通过MailBox通讯，共享内存交换数据。

HPMCU具有400MHz的主频，提供了较强的计算能力，能够快速响应低延时任务。

产品版本

芯片名称	**RT-Thread**版本
RV1126B	V4.1.x

读者对象

本文档（本指南）主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	ZSQ	2025-06-05	初始版本

目录

Rockchip RV1126B RT-Thread MCU 快速开发指南
1. 服务器环境搭建
2. 获取RV1126B Linux SDK包
3. RT-Thread相关目录结构
4. MCU支持的功能
5. 编译说明
   5.1 逐步编译MCU RT-Thread（适合开发调试）
   5.2 （打包方式一）独立打包
   5.3 （打包方式二）打包到uboot.img中
   5.4 在Linux SDK中编译打包（适合固件发布）
6. 初次使用
   6.1 固件烧录
   6.2 使用串口连接到RT-Thread
   6.3 MCU串口MSH界面
   6.4 修改defconfig配置及保存
   6.5 添加一个板级配置
   6.6 MCU与Kernel的资源分配
      6.6.1 Kernel DTS 引用rv1126b-mcu.dtsi
   6.7 使用JTAG调试

1. 服务器环境搭建

推荐使用 Ubuntu 20.04及以上的系统进行编译，其他的Linux版本可能需要对信赖的软件包做相应调整。除了系统要求外，还有其他软硬件方面的要求。

硬件要求：64位系统，硬盘空间大于20G。如果您进行多个构建，将需要更大的硬盘空间。

软件要求：Ubuntu 20.04及以上

编译RT-Thread所依赖的软件包安装命令如下：

```
# Notice: use python2 default
sudo apt-get update --fix-missing

sudo apt-get install repo git scons python-pip

pip install numpy
pip install configparser
```

若编译遇到软件依赖报错，可以视报错信息安装对应的软件包。

2. 获取RV1126B Linux SDK包

请联系业务或FAE获取对应芯片的Linux SDK Release文档。

RV1126B MCU的RT-Thread及HAL工程代码是跟随RV1126B SDK一同发布的。二者在编译打包、资源分配上有一定的互斥、依赖关系。

3. RT-Thread相关目录结构

目录路径	说明
prebuilts/gcc/linux-x86/riscv	交叉工具链
rtos	RT-Thread代码
rtos/bsp/rockchip/common/hal hal	RT-Thread的HAL层代码 (Hardware Abstract Layer)
rtos/bsp/rockchip/common/drivers	Rockchip相关主要驱动代码
rtos/bsp/rockchip/common/tests	Rockchip相关主要测试、Demo代码
docs/rtt-docs	相关文档

4. MCU支持的功能

模块	**Defconfig**选项	**Demo**代码
CLK	CONFIG_RT_USING_CRU	应用一般不直接调用clk
GPIO	CONFIG_RT_USING_PIN CONFIG_RT_USING_COMMON_TEST_GPIO_V2	tests/gpio_v2_test.c board/common/iomux_base.c
PWM	CONFIG_RT_USING_PWM CONFIG_RT_USING_PWM[0~3] CONFIG_RT_USING_PWM_REMOTECTL_TEST	使用rtt的驱动框架，可参考： /rtos/components/drivers/misc/rt_drv_pwm.c drivers/drv_backlight.c
SARADC	CONFIG_RT_USING_ADC CONFIG_RT_USING_MULTI_SARADC CONFIG_RT_USING_SARADC[0~2]	使用rtt的驱动框架，可参考： /rtos/components/drivers/misc/adc.c
UART	CONFIG_RT_USING_UART CONFIG_RT_USING_UART[0~7]	可参考调试串口的输入输出： /rtos/src/kservice.c
I2C	CONFIG_RT_USING_I2C CONFIG_RT_USING_I2C[0~5]	tests/i2c_test.c
RPMSG	CONFIG_RT_USING_RPMSG_LITE CONFIG_RT_USING_LINUX_RPMSG CONFIG_RT_USING_COMMON_TEST_LINUX_RPMSG_LITE	tests/rpmsg_test.c
Timer	RT_USING_COMMON_TEST_TIMER TIMER0	tests/timer_test.c

- Demo代码位于rtos/bsp/rockchip/common或rtos/bsp/rockchip/rv1126b-mcu目录下

5. 编译说明

编译分为编译、打包两个部分。

- 编译，编译生成rtthread.bin rtthread.elf rtthread.map
- 打包，以FIT格式打包rthtread.bin，生成最终可烧录的固件，如amp.img。

RV1126B MCU适配在多个产品上，有些需要快速启动，有些关注灵活升级。用户可以根据自己的产品需求相应选择不同的打包方式。

编译方式	优点	适用场景
逐步执行编译命令	单独编译，可调整编译的步骤、配置	MCU开发验证
使用Linux SDK一键编译打包	不关心MCU的具体编译细节，一键编译及打包	固件的发布、定期编译

在RV1126B Linux SDK中， 使用Linux SDK一键编译打包的方式编译时，默认是采用独立打包amp.img的方式进行打包。如需要修改成打包到uboot.img中，SDK暂未支持，用户可自行参照修改。

打包方式	优点	适用场景
独立打包amp.img	1. 在Uboot阶段加载 2. 不依赖Uboot的编译 3 有独立的一个分区(amp)，OTA更方便	MCU启动时间不在乎多100~200毫秒 需要单独升级MCU固件
打包到uboot.img中	1. 在SPL阶段加载，更快启动MCU 2. 与uboot.img一起做固件的签名、校验	对MCU启动时间有较高要求 对MCU有固件签名需求

5.1 逐步编译MCU RT-Thread（适合开发调试）

以RV1126B EVB1为例，进入到对应的bsp目录，使用`scons`加载Config并编译：

```
$ cd rtos/bsp/rockchip/rv1126b-mcu

$ scons --useconfig=board/rv1126b_evb1/defconfig
$ scons --genconfig
Generate .config done!
$ scons -j16
....
LINK rtthread.elf
riscv-none-embed-objcopy -O binary rtthread.elf rtthread.bin
riscv-none-embed-size rtthread.elf
   text    data     bss     dec     hex filename
 108935    2872  256440  368247   59e77 rtthread.elf
scons: done building targets.
```

编译生成`rtthread.bin`及符号表：

```
$ ls rtthread.*
rtthread.bin  rtthread.elf  rtthread.map
```

- 注：上述三个文件还无法直接烧录到板上， 需要进行打包成合适的Image。

5.2 （打包方式一）独立打包

将`rthtread.bin`以FIT格式打包成一个`amp.img`。

```
# 根据Image/amp.its打包
$ ./mkimage.sh amp
# 输出文件为amp.img
$ file -L Image/amp.img
Image/amp.img: Device Tree Blob version 17, size=1024, boot CPU=0, string block size=196, DT
structure block size=616
```

- 可通过Rockchip开发工具将`amp.img`烧录到开发板的`amp`分区

5.3 （打包方式二）打包到uboot.img中

将rtthread.bin拷贝到rkbin目录下，并最终打包在uboot.img中。

```
$ cd rtos/bsp/rockchip/rv1126b-mcu
$ cp rtthread.bin <YOUR_RV1126B_LINUX_SDK_PATH>/rkbin/bin/rv11/rv1126b-mcu.bin
```

在Uboot Config中，指定`CONFIG_TRUST_INI`为`RV1126BTRUST_MCU.ini`， 该文件有声明MCU的固件名称及其加载地址。

编译并打包uboot.img：

```
$ cd u-boot
$ echo 'CONFIG_TRUST_INI="RV1126BTRUST_MCU.ini"' >> configs/rv1126b_defconfig
$ ./make.sh rv1126b --spl-new
# (可选) 检查打包是否成功
$ fdtdump uboot.img | grep -A 5 mcu
...
        mcu0 {
            data-size = <0x0000d834>;
            data-position = <0x00055600>;
            description = "mcu0";
            type = "standalone";
            arch = "riscv";
            load = <0x48c02000>;
            compression = "lzma";
            digest {
...
```

使用烧录工具分别将`rv1126b_spl_loader_v1.xxx.bin`及`uboot.img`烧录到EVB板子上。

5.4 在Linux SDK中编译打包（适合固件发布）

RV1126B Linux SDK集成了MCU编译的功能，能够方便集成编译MCU固件，并打包到生产或OTA 的update.img中。

RV1126B Robot Defconfig默认使能了MCU，以该配置为例：

```
# 选择平台
$ ./build.sh chip
...
21. rk3588
22. rv1126_rv1109
23. rv1126b
Which would you like? [1]: 23
Switching to chip: rv1126b
Pick a defconfig:

1. rockchip_defconfig
2. rockchip_rv1126b_evb1_v10_defconfig
3. rockchip_rv1126b_ipc_32_evb1_v10_defconfig
4. rockchip_rv1126b_ipc_64_evb1_v10_defconfig
5. rockchip_rv1126b_robot_defconfig
...
Which would you like? [1]: 5
Switching to defconfig:
/home/zsq/29/rv1126b/device/rockchip/.chip/rockchip_rv1126b_robot_defconfig
#
# configuration written to /home/zsq/29/rv1126b/output/.config
#
```

编译并生成amp.img：

```
$ ./build.sh amp
############### Rockchip Linux SDK ###############
Manifest: rv1126b_linux.xml
...
RK_AMP=y
RK_AMP_ARCH=riscv
RK_AMP_FIT_ITS=amp_mcu.its
RK_AMP_HAL_TARGET=rv1126b
RK_AMP_MCU_HAL_TARGET=rv1126b-mcu
RK_AMP_MCU_RTT_TARGET=rv1126b-mcu
RK_AMP_RISCV=y
RK_AMP_RTT_TARGET=rv1126b-32
...
Image info: mcu
FIRMWARE_CPU_BASE=0x48c02000
DRAM_SIZE=0x0003a000
SRAM_BASE=
SRAM_SIZE=
SHMEM_BASE=
```

```
LINUX_RPMSG_BASE=0x48c3c000
LINUX_RPMSG_SIZE=0x00040000
...
Running 25-amp.sh - amp amp succeeded.

# 编译输出
$ ls output/firmware/amp.img
output/firmware/amp.img
```

如果其它板级配置需要使能MCU，可参考：

- device/rockchip/rv1126b/rockchip_rv1126b_robot_defconfig
- device/rockchip/rv1126b/amp_mcu.its，修改相应的rtt_config以选择RT-Thread的板级文件

6. 初次使用

6.1 固件烧录

建议用户先完整烧录一次Linux固件（一般是一个`update.img`），Linux SDK编译时板级需要选择在Linux SDK中编译打包小节中描述的配置，该配置有声明开启MCU的加载。这是因为Linux Kernel与MCU间存在资源共享互斥，且引导系统（默认UBoot，SPL也能支持）需要能够加载MCU固件。

如果您只关注MCU的开发、尚不清楚如何烧录Linux固件，请参考Linux SDK中的文档，如发布说明、快速开发指南等，或向负责Linux BSP的同事寻找帮助。例如：

```
# 先进入Maskrom模式
$ sudo upgrade_tool uf <Linux SDK Path>/output/firmware/update.img
```

如果想要烧录`amp.img`，有灵活的多种方法。

方式一，USB线连接到EVB1板上的`USB3.0_HOST`，长按`UPDATE`按键并上电，可进入Maskrom模式下烧录，

```
# 先进入Maskrom模式，并烧入Loader
$ sudo upgrade_tool output/firmware/MiniLoaderAll.bin -noreset
# 烧录amp.img
$ sudo upgrade_tool di -amp apm.img
# 复位系统
$ sudo upgrade_too rd
```

- `upgrade_tool`工具位于/tools/linux/Linux_Upgrade_Tool目录下，访问USB需要超级用户权限
- Windows版本的开发工具请使用/tools/windows/RKRKDevTool_Release_xxx.zip

方式二，USB线连接到EVB1板上的`USB3.0_HOST`，开机后进入到Linux SDK，能够识别到`adb`，

```
$ adb shell
# 进入到板端的命令行，重启进入Loader模式
# reboot loader

# 烧录amp.img
$ sudo upgrade_tool di -amp apm.img
# 复位系统
$ sudo upgrade_too rd
```

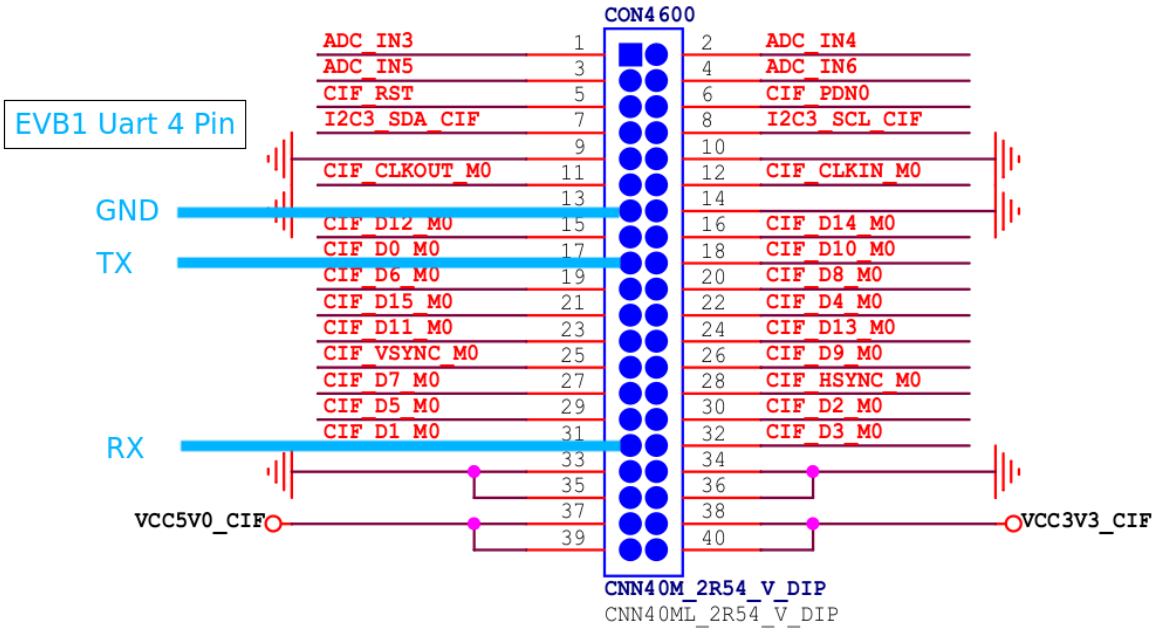
方式三，USB线连接到EVB1板上的`USB3.0_HOST`，开机后进入到Linux SDK，能够识别到`adb`，

```
$ adb push amp.img /tmp/
$ adb shell
# 进入到板端的命令行，直接写入分区并重启
# dd if=/tmp/amp.img of=/dev/block/by-name/amp bs=1M oflag=dsync
# reboot
```

- 注意，该方式不适用于Nand Flash存储，只适用于配置了EMMC的板子

6.2 使用串口连接到RT-Thread

如下图所示， RV1126B EVB1 默认使用UART4_M2(GPIO6_A0、GPIO6_A1)作为MCU调试串口。 在EVB1板上，找到GPIO(CIF/ADC)双排插针，如下线序示意图（注意板上Pin 1引脚位置）：



6.3 MCU串口MSH界面

UART4 串口波特率为1500000。有些串口小板无法支持1.5Mbps的波特率，参考修改为115200：

```
--- a/bsp/rockchip/rv1126b-mcu/board/common/board_base.c
+++ b/bsp/rockchip/rv1126b-mcu/board/common/board_base.c
@@ -76,7 +76,7 @@ RT_WEAK const struct uart_board g_uart2_board =
 #if defined(RT_USING_UART4)
 RT_WEAK const struct uart_board g_uart4_board =
 {
-    .baud_rate = UART_BR_1500000,
+    .baud_rate = UART_BR_115200,
     .dev_flag = ROCKCHIP_UART_SUPPORT_FLAG_DEFAULT,
     .bufer_size = RT_SERIAL_RB_BUFSZ,
     .name = "uart4",
```

上电后，可以看到UART4串口界面：

```
msh />
 \ | /
- RT -     Thread Operating System
 / | \     4.1.1 build Jun  5 2025 10:45:23
 2006 - 2022 Copyright by RT-Thread team
Hello RV1126B MCU
msh />
```

使用help可查看命令使用说明：

```
msh />help
```

注意：上电后因为Linux Kernel也会同步引导，因此：

- MCU与Linux双系统需要预先分配好资源，参考资源分配小节，或者
- 如果只关注MCU调试，在Uboot引导阶段，中止引导Kernel也不会影响MCU的运行

中止引导Kernel，例如：在开机过程中，在Uboot的串口输出终端(注意，这里指的的ARM A53核心的串口，即EVB1的UART0串口)长按Ctrl+C按键，

```
...
U-Boot 2017.09-g25463d45566-241231 #zhengsq (Jun 05 2025 - 08:14:14 +0800)
...
Hotkey: ctrl+c
...
Net:   eth0: ethernet@21c70000
Handle standalone: 'hpmcu' at 0x48c02000 ...Starting hpmcu at 0000000048c02000
OK
Hit key to stop autoboot('CTRL+C'):  0
=> <INTERRUPT>
=> <INTERRUPT>
...
```

6.4 修改defconfig配置及保存

使用`scons`加载Defconfig，并修改配置，例如：

```
$ cd rtos/bsp/rockchip/rv1126b-mcu

# 加载evb1的defconfig配置，修改到rtconfig.h文件中
$ scons --useconfig=board/rv1126b_evb1/defconfig

# 根据rtconfig.h生成.config文件
$ scons --genconfig
scons: Reading SConscript files ...
Generate .config done!

# 使用menuconfig修改配置，修改到.config及rtconfig.h文件中
$ scons --menuconfig

# 生成更简洁的.config， 并拷贝保存在defconfig中
$ scons --genconfig
$ cp .config board/rv1126b_evb1/defconfig
```

6.5 添加一个板级配置

建议为项目增加一个新的板级配置， 例如参考`board/rv1126b_evb1`拷贝一份：

```
$ cd rtos/bsp/rockchip/rv1126b-mcu
$ cp -r board/rv1126b_evb1 board/rv1126b_robot
```

根据项目信息， 修改iomux配置、使用所需的外设（UART、I2C、PWM、ADC等）

6.6 MCU与Kernel的资源分配

MCU及Linux Kernel二者需要互斥地访问资源， 包括CLK、DDR、GPIO、ADC、PWM等外设接口。

在Kernel DTS中， 在`rv1126b-mcu.dtsi`节点中声明MCU要使用到的DDR，CLK，Pinctrl，以及关闭相应的外设控制器。

举例说明如下。

6.6.1 Kernel DTS 引用rv1126b-mcu.dtsi

在项目的板级DTS中，声明引用`rv1126b-mcu.dtsi`。例如在RV1126B EVB1的DTS中，

```
#include "rv1126b-mcu.dtsi"
```

RT-Thread默认跑在DDR上，因此Kernel需要声明预留MCU使用的内存`hpmcu_reserved`，该地址为固定值，不要修改。

```
/ {
        reserved-memory {
                /* hpmcu address */
                hpmcu_reserved: hpmcu@48c02000 {
                        reg = <0x48c02000 0x3a000>;
                        no-map;
                };
        };
};
```

- 注意，该地址0x48c02000是MCU的入口地址，下述文件中该地址应保持一致：
  - `link.lds`，即`rtos/bsp/rockchip/rv1126b-mcu/link.lds`，逐步执行编译命令编译RT-Thread时指定链接地址
  - `amp_mcu.its`，即`device/rockchip/rv1126b/amp_mcu.its`，使用Linux SDK一键编译打包时引用到
  - `Image/amp.its`，即`rtos/bsp/rockchip/rv1126b-mcu/Image/amp.its`，独立打包amp.img时引用到
  - `RV1126BTRUST_MCU.ini`，即`rkbin/RKTRUST/RV1126BTRUST_MCU.ini`，打包到uboot.img中时引用到

如果需要新增MCU对外设的访问，例如，UART4用于MCU的串口调试功能，那么需要声明UART4的CLK、Pinctrl信息，表示该UART4已经被MCU使用， 让Kernel不关闭它的CLK，也不要有Pinctrl设置上的冲突。

```
/ {
        rockchip_amp: rockchip-amp {
                compatible = "rockchip,amp";
                clocks = <&cru TCLK_WDT_HPMCU>, <&cru PCLK_WDT_HPMCU>,
                                <&cru HCLK_CACHE>, <&cru CLK_HPMCU>,
                                <&cru CLK_HPMCU_RTC>, <&cru PCLK_HPMCU_INTMUX>,
                                <&cru PCLK_HPMCU_MAILBOX>,
                                <&cru SCLK_UART4>, <&cru PCLK_UART4>;
                pinctrl-names = "default";
                pinctrl-0 = <&uart4m2_xfer_pins>;
                status = "okay";
        };
};

&uart4 {
        status = "disabled";
};
```

- 声明了UART4所需的两个CLK：`SCLK_UART4`、`PCLK_UART4`
- 声明了UART4实际使用的PIN：`uart4m2_xfer_pins`
- 还明确将UART4设置为`disabled`， 让Kernel不去访问UART4

以此类推， 如果需要其它外设， 也要声明对应的CLK、Pinctrl，在Kernel端关闭该外设。

同时，Kernel的Config配置， 也要加上Rockchip AMP，即：`arch/arm64/configs/rockchip_amp.config`，主要有（没有全部列出）：

```
CONFIG_MAILBOX=y
CONFIG_ROCKCHIP_AMP=y
CONFIG_RPMSG_VIRTIO=y
CONFIG_ROCKCHIP_MBOX=y
CONFIG_RPMSG=y
```

6.7 使用JTAG调试

RV1126B RISCV MCU支持JTAG调试，使用OpenOCD及GDB可连接到板端；该功能默认关闭。如需要开启，请注意在Release固件中要关掉JTAG的使能。

获取OpenOCD及开启JTAG的详细方法，请联系Rockchip技术窗口。